

Multicommodity Flow Problem

Lagrange methods

David Pisinger

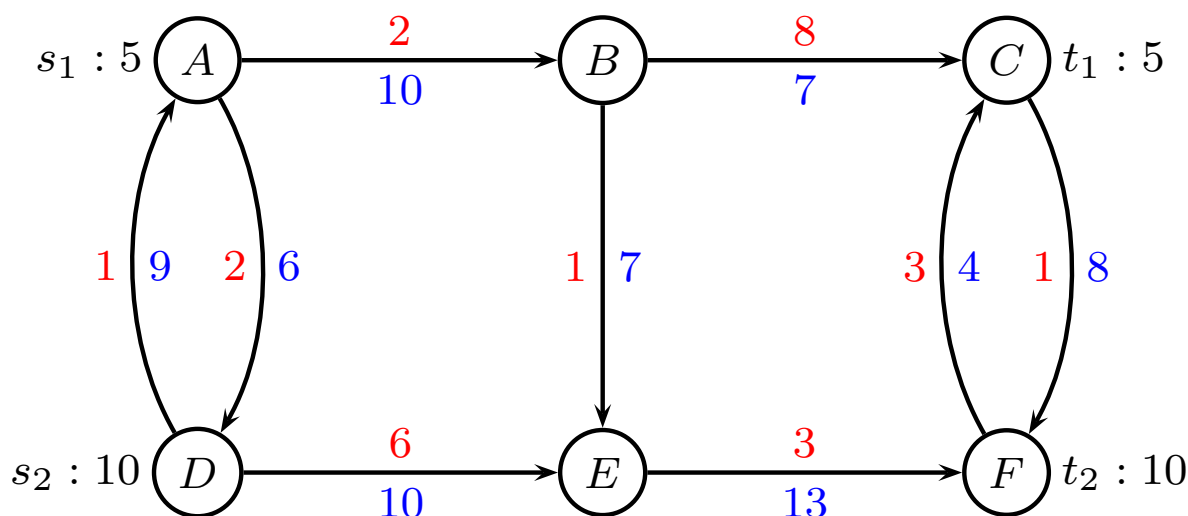
`pisinger@man.dtu.dk`

DTU Management
Technical University of Denmark

© Copyright David Pisinger. May not be distributed

Multicommodity Flow - Terminology

- We consider a directed graph $G = (V, E)$ in which each edge e has a capacity $u_e \in \mathbb{R}_+$ and a unit transportation cost $c_e \in \mathbb{R}$.
- A set K of commodities is given. Each commodity k has a source s_k , terminal t_k , and demand d_k .
- All commodities k should be transported from source to sink in the cheapest possible way, respecting capacities.



Multicommodity Flow - Definition

Let x_{ij}^k denote flow of commodity k on edge (i, j) .

$$\begin{aligned} \min \quad & \sum_{k \in K} \sum_{(i,j) \in E} c_{ij} x_{ij}^k \\ \text{s.t.} \quad & \sum_{i \in V} x_{ij}^k - \sum_{i \in V} x_{ji}^k = 0 \quad k \in K, j \in V \setminus \{s_k, t_k\} \\ & \sum_{i \in V} x_{ij}^k - \sum_{i \in V} x_{ji}^k = -d_k \quad k \in K, j = s_k \\ & \sum_{i \in V} x_{ij}^k - \sum_{i \in V} x_{ji}^k = d_k \quad k \in K, j = t_k \\ & \sum_{k \in K} x_{ij}^k \leq u_{ij} \quad (i, j) \in E \\ & 0 \leq x_{ij}^k \leq d_k \quad k \in K, (i, j) \in E \end{aligned}$$

Bounds $x_{ij}^k \leq d_k$ not strictly necessary.

MCF - compact formulation

Multicommodity flow problem

$$\begin{aligned} \min \quad & \sum_{k \in K} \sum_{(i,j) \in E} c_{ij} x_{ij}^k \\ \text{s.t.} \quad & \sum_{i \in V} x_{ij}^k - \sum_{i \in V} x_{ji}^k = b_j^k \quad k \in K, j \in V \quad (\pi_j^k) \\ & \sum_{k \in K} x_{ij}^k \leq u_{ij} \quad (i,j) \in E \quad (y_{ij}) \\ & x_{ij}^k \geq 0 \quad k \in K, (i,j) \in E \end{aligned}$$

where demand b_j^k in each node is

$$b_j^k = \begin{cases} 0 & \text{if } j \neq \{s_k, t_k\} \\ d_k & \text{if } j = t_k \\ -d_k & \text{if } j = s_k \end{cases}$$

Solution methods

LP-based

- Simplex
- interior-point methods

Column generation

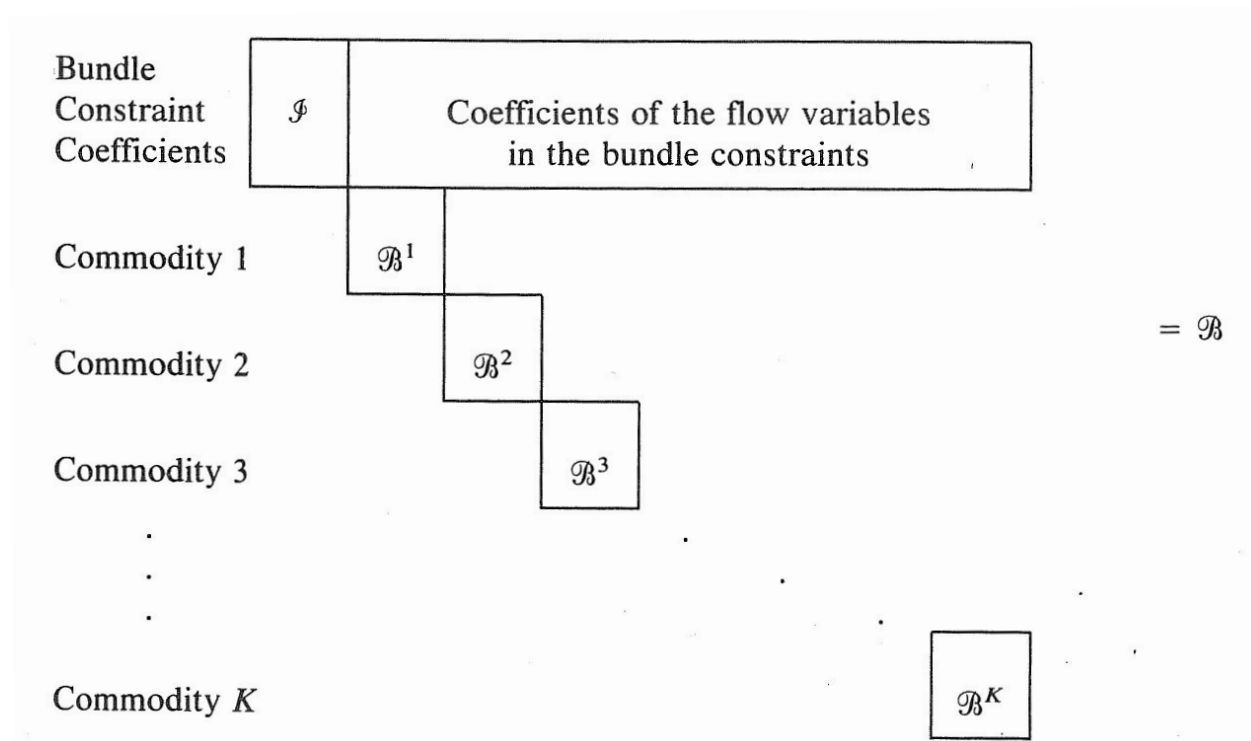
- Dantzig Wolfe decomposition
- Master Problem
- Subproblem

Lagrangian methods

- Relax (delete) capacity constraints
- Punish costs if capacity violated

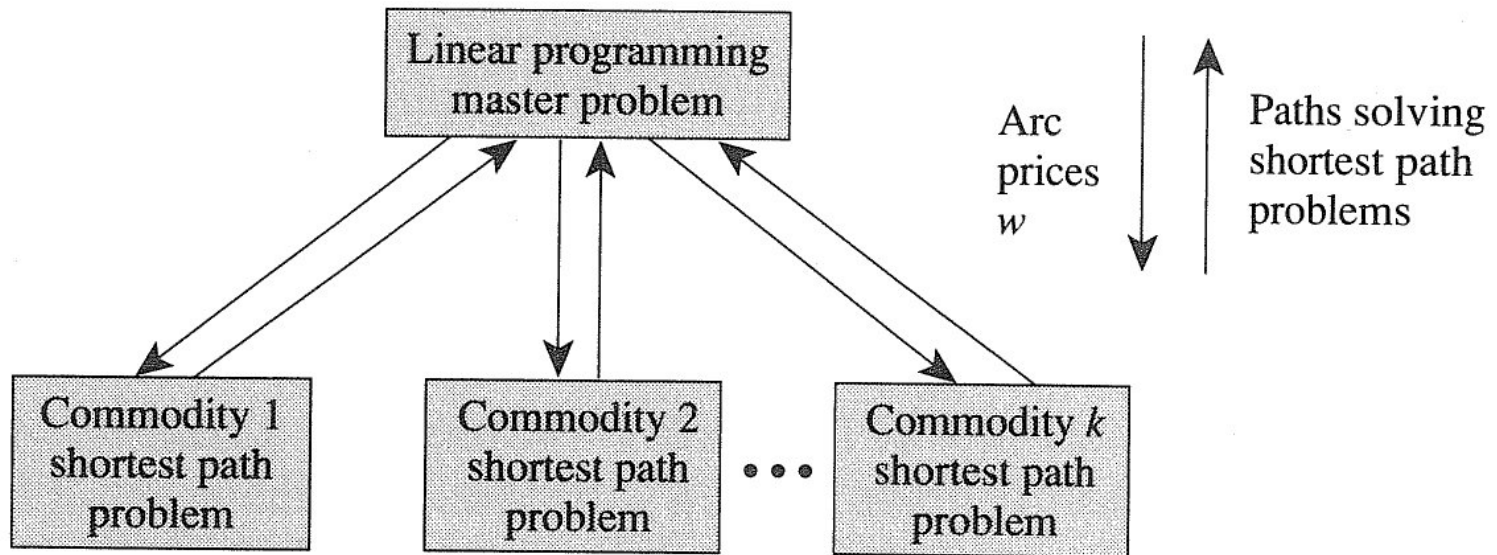
Dantzig Wolfe in general

The problem has block angular structure



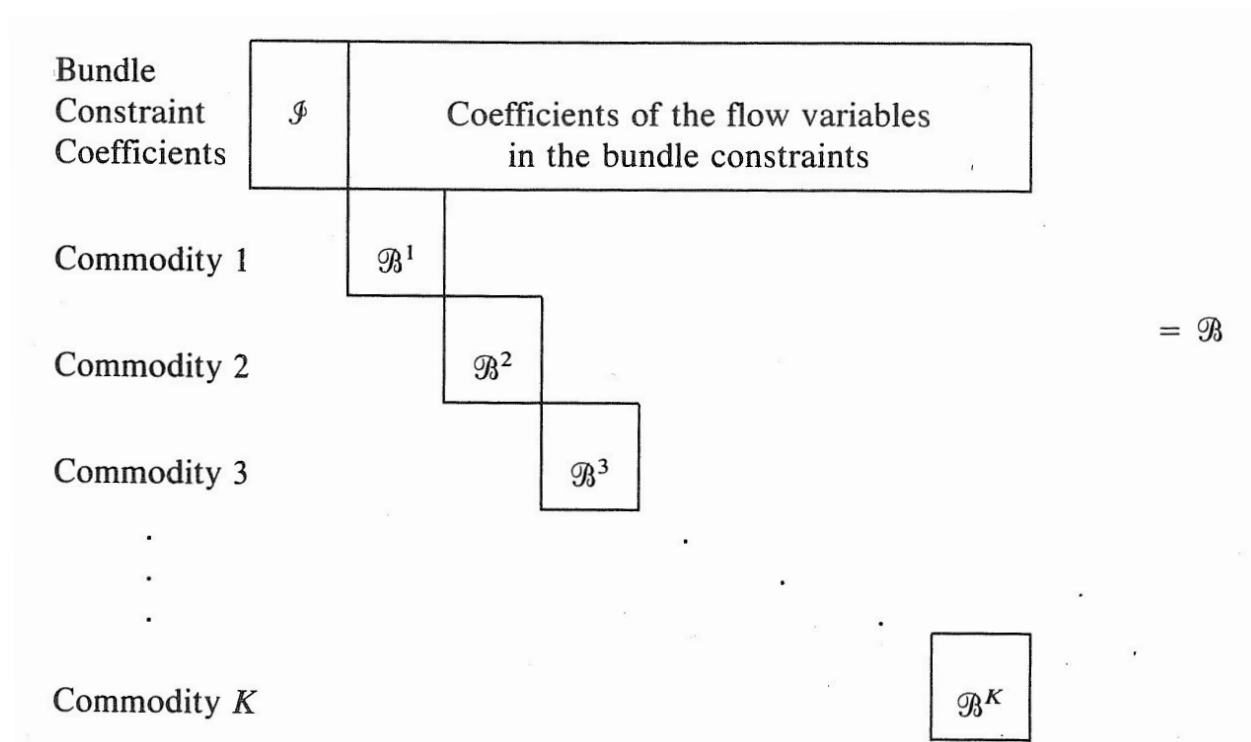
Reformulate by writing up all solutions to each block
Since reformulation may be big, write up as needed
Delayed column generation

Delayed column generation



Another idea

Get rid of bundle constraints (capacity constraints)



Problem becomes independent flow problem for each k

Some theory - Lagrangian relaxation

$$\begin{array}{ll}\text{minimize} & cx \\ \text{subject to} & Ax \leq b \\ & Dx \leq d \\ & x_j \in \mathbb{Z}_+, \quad j = 1, \dots, n\end{array}$$

Lagrange relax $Dx \leq d$, using multipliers $\lambda \geq 0$

$$\begin{array}{ll}\text{minimize} & z_{LR}(\lambda) = cx + \lambda(Dx - d) \\ \text{subject to} & Ax \leq b \\ & x_j \in \mathbb{Z}_+, \quad j = 1, \dots, n\end{array}$$

Some theory - Lagrangian relaxation

Proposition Optimal solution to relaxed problem is lower bound on original problem

Proof It is a relaxation

1. Set of feasible solutions is extended
2. Lagrangian relaxed objective function is smaller

multiplier λ_i “punishment”

If λ_i large $\Rightarrow$ constraint satisfied

If $\lambda_i = 0 \Rightarrow$ drop constraint

Some theory - Lagrangian relaxation

Lagrange relaxed problem as function of $\lambda \geq 0$

$$\begin{array}{ll}\text{minimize} & z_{LR}(\lambda) = cx + \lambda(Dx - d) \\ \text{subject to} & Ax \leq b \\ & x_j \in \mathbb{Z}_+, \quad j = 1, \dots, n\end{array}$$

Lagrangian Dual Problem

$$z_{LD} = \max_{\lambda \geq 0} z_{LR}(\lambda)$$

Find the λ that gives the largest lower bound

Lagrangian relaxation, MCFP

$$\begin{aligned} \min \quad & \sum_{k \in K} \sum_{(i,j) \in E} c_{ij} x_{ij}^k + \sum_{(i,j) \in E} \lambda_{ij} (\sum_{k \in K} x_{ij}^k - u_{ij}) \\ \text{s.t.} \quad & \sum_{i \in V} x_{ij}^k - \sum_{i \in V} x_{ji}^k = b_j^k \quad k \in K, j \in V \\ & x_{ij}^k \geq 0 \quad k \in K, (i,j) \in E \end{aligned}$$

Or equivalently

$$\begin{aligned} \min \quad & \sum_{k \in K} \sum_{(i,j) \in E} (c_{ij} + \lambda_{ij}) x_{ij}^k - \sum_{(i,j) \in E} \lambda_{ij} u_{ij} \\ \text{s.t.} \quad & \sum_{i \in V} x_{ij}^k - \sum_{i \in V} x_{ji}^k = b_j^k \quad k \in K, j \in V \\ & x_{ij}^k \geq 0 \quad k \in K, (i,j) \in E \end{aligned}$$

Last sum in objective is a "constant".

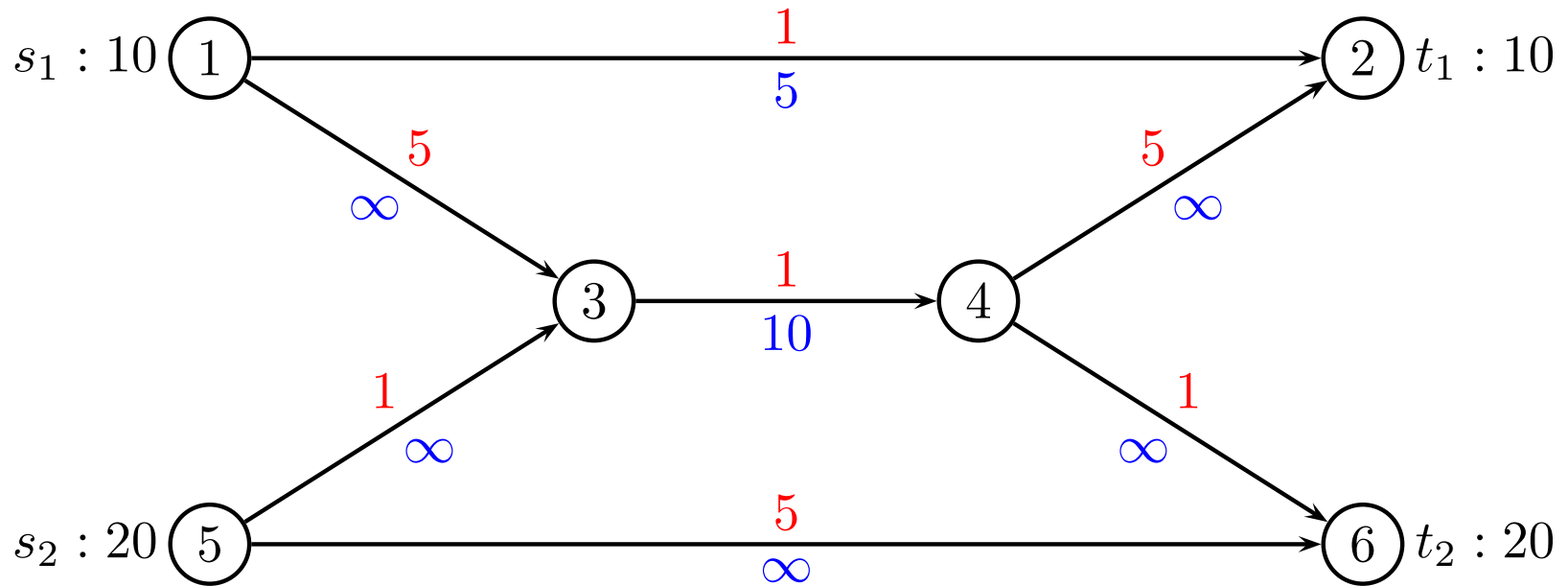
Lagrangian relaxation

Since no shared capacity, splits into separate problems for each $k \in K$

$$\begin{aligned} \min \quad & \sum_{(i,j) \in E} (c_{ij} + \lambda_{ij}) x_{ij}^k \\ \text{s.t.} \quad & \sum_{i \in V} x_{ij}^k - \sum_{i \in V} x_{ji}^k = b_j^k \quad j \in V \\ & x_{ij}^k \geq 0 \quad (i, j) \in E \end{aligned}$$

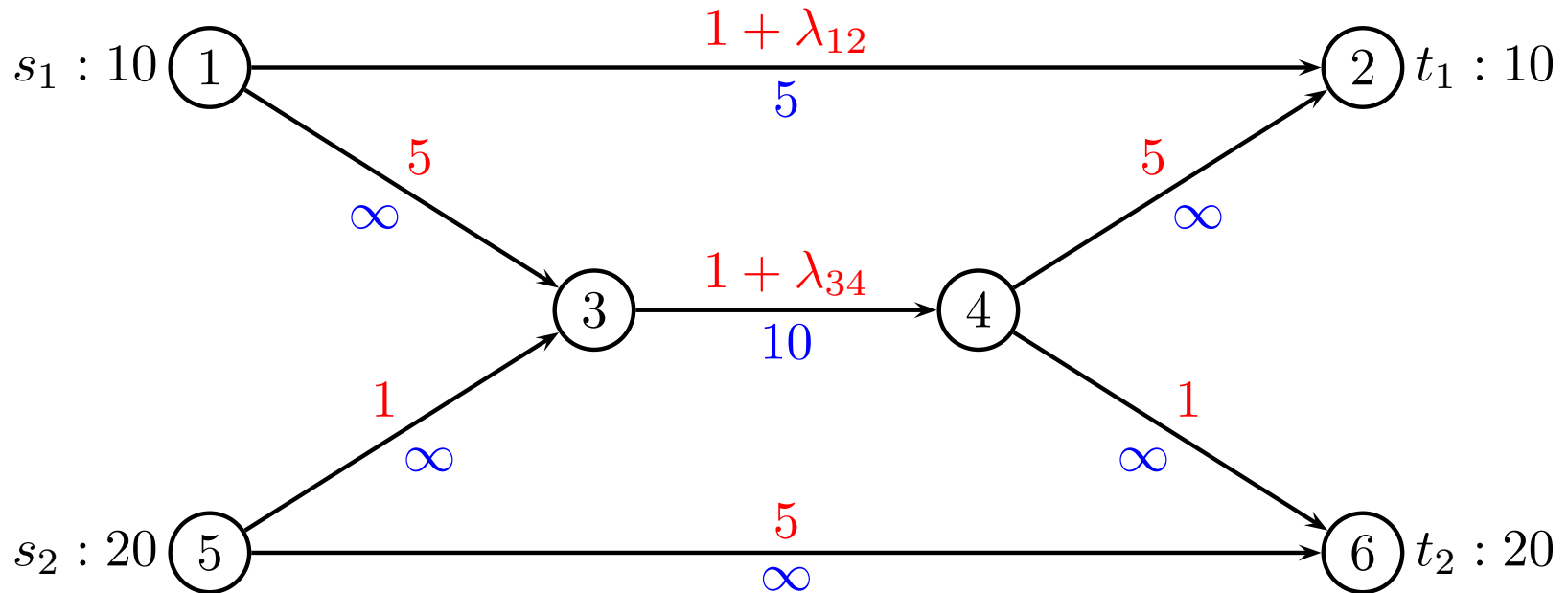
This is a minimum cost flow problem for each k
(actually a shortest path problem)

Multicommodity Flow - example



Multicommodity Flow - example

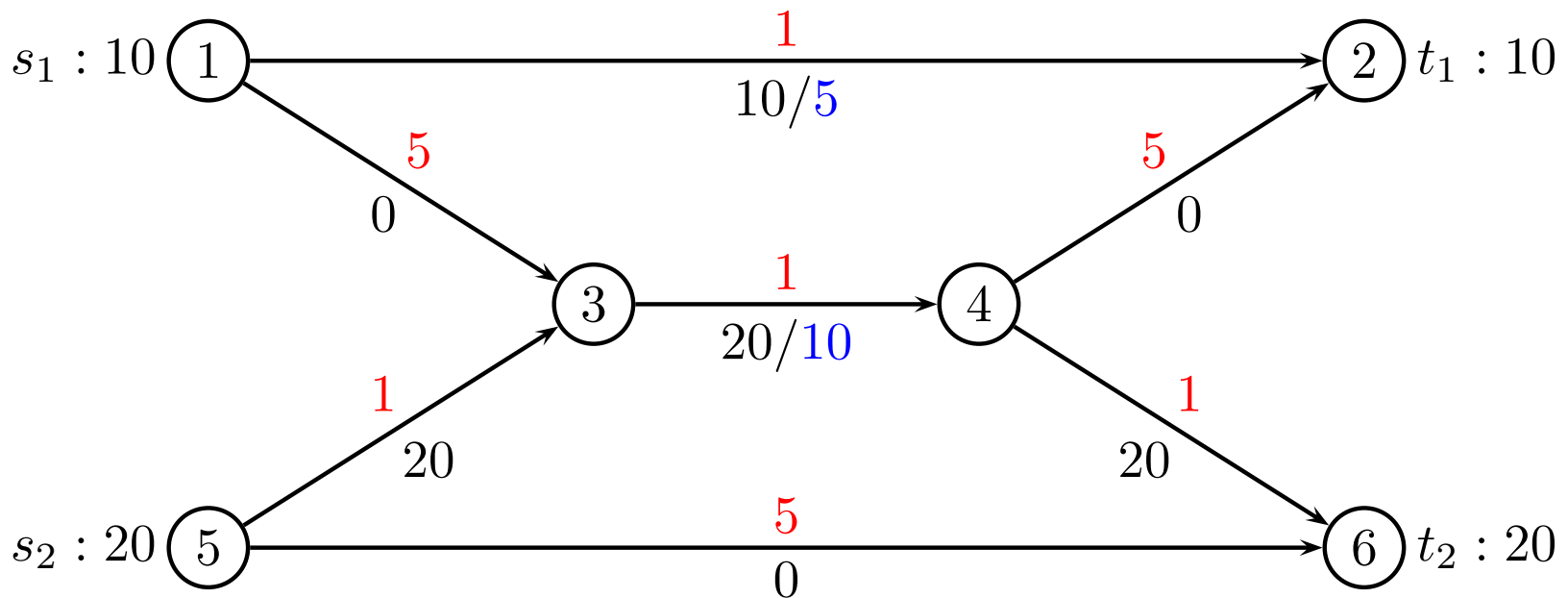
Lagrange relaxed



Iteration 0

Initially choose all Lagrange multipliers $\lambda = 0$

$$\lambda_{12} = 0, \lambda_{34} = 0$$



flow cost: $10 \cdot (1) + 20 \cdot (1 + 1 + 1) = 70$

constant: $5\lambda_{12} + 10\lambda_{34} = 0$

lower bound: 70

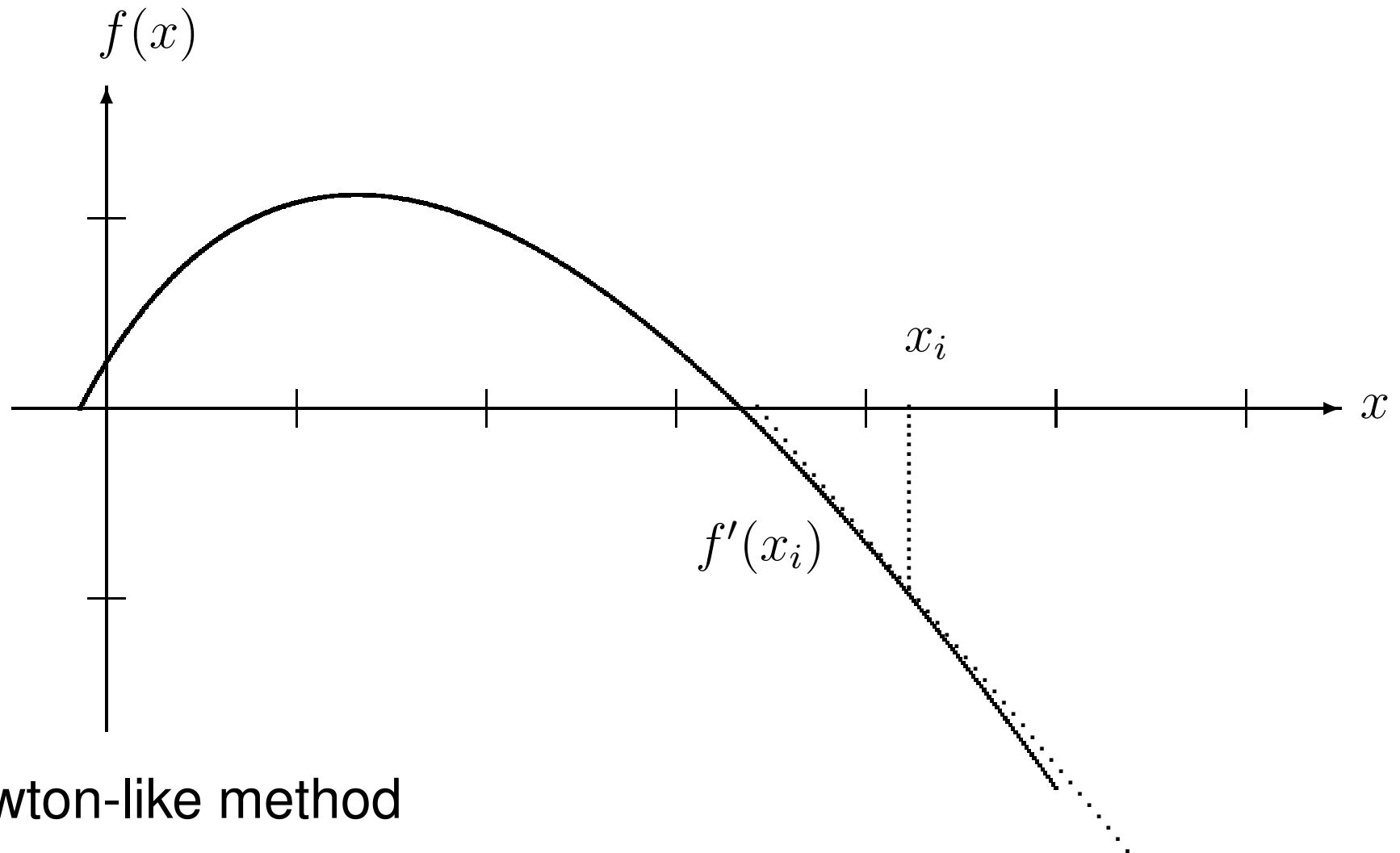
How can we improve this?

Find the λ that gives the largest lower bound
Lagrange Dual Problem

$$z_{LD} = \max_{\lambda \geq 0} z_{LR}(\lambda)$$

- Often we do not need best multipliers
- Subgradient optimization fast method
- Methods for minimizing a convex, possibly non-differentiable, function over a convex and closed set
- Roots in nonlinear programming, Held and Karp (1971)

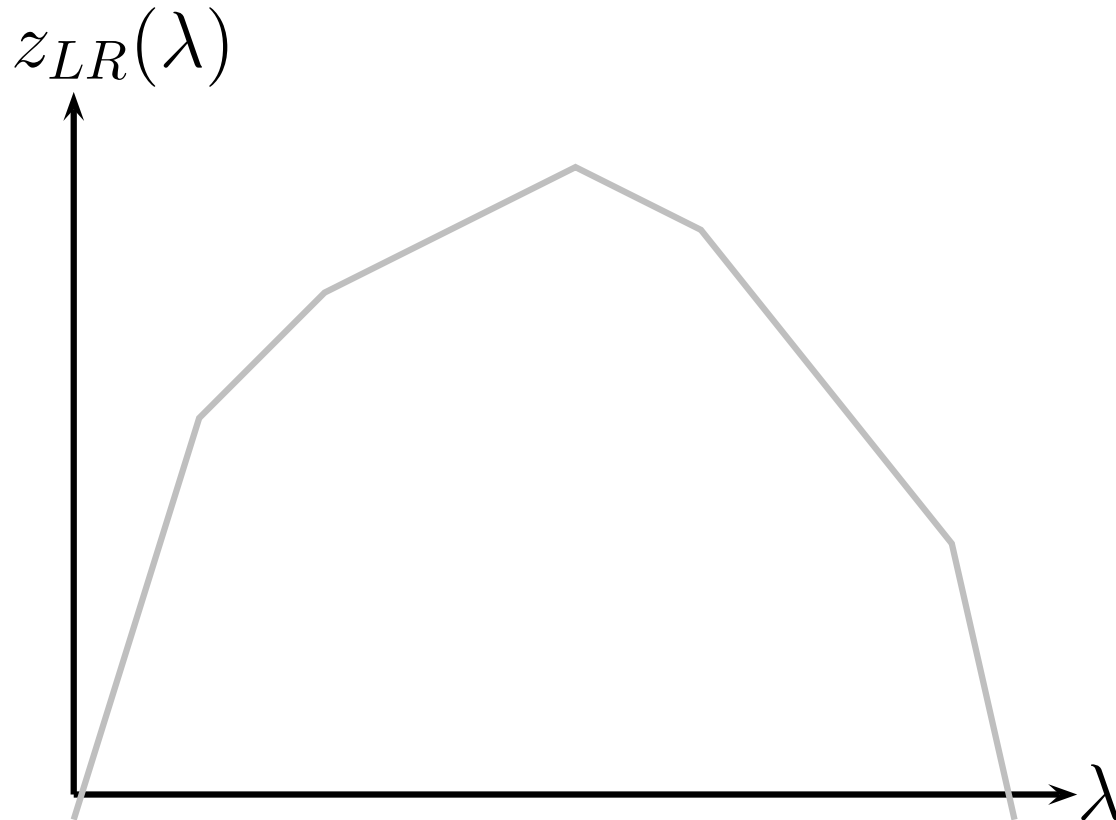
How can we improve this?



Newton-like method

How can we improve this?

Lagrange function $z_{LR}(\lambda)$ is piecewise linear and convex

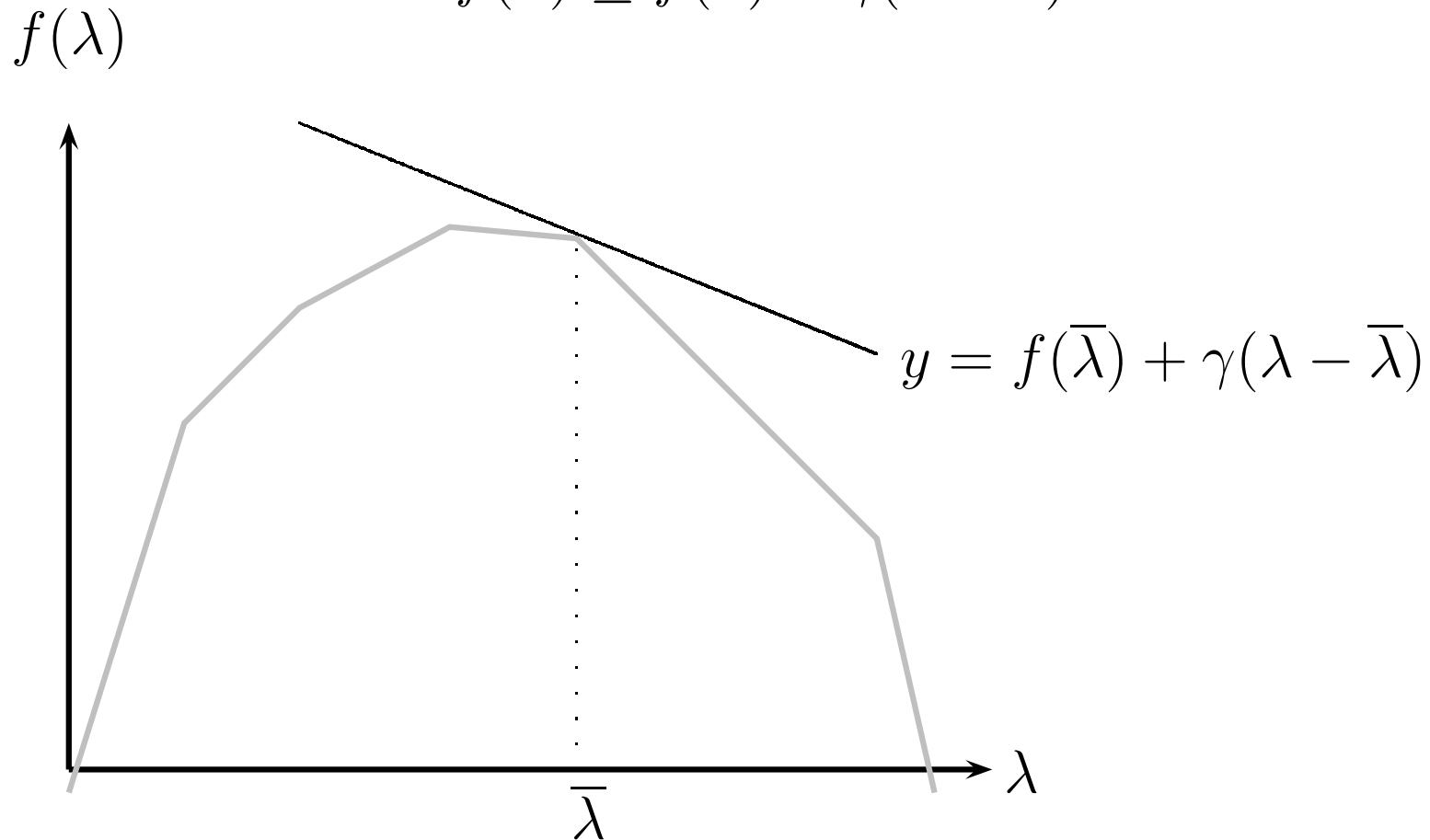


Subgradient

Generalization of gradients to non-differentiable functions.

Definition An m -vector γ is subgradient of $f(\lambda)$ at $\lambda = \bar{\lambda}$ if

$$f(\lambda) \leq f(\bar{\lambda}) + \gamma(\lambda - \bar{\lambda})$$



Subgradient

Given a choice of nonnegative multipliers $\bar{\lambda}$. If x' is an optimal solution to $z_{LR}(\bar{\lambda})$ then

$$\gamma = Dx' - d$$

is a subgradient of $z_{LR}(\lambda)$ at $\lambda = \bar{\lambda}$.

Intuition: Lagrange Relaxation

$$\begin{array}{ll}\text{minimize} & z_{LR}(\lambda) = cx + \lambda(Dx - d) \\ \text{subject to} & Ax \leq b \\ & x_j \in \mathbb{Z}_+, \quad j = 1, \dots, n\end{array}$$

want to solve

$$z_{LD} = \max_{\lambda \geq 0} z_{LR}(\lambda)$$

For small changes of λ we find same x' so γ is subgradient

Subgradient iteration

Recursion

$$\lambda^{(k+1)} = \max \left\{ \lambda^{(k)} + \theta \gamma^{(k)}, 0 \right\}$$

Where $\theta > 0$ is step-size.

If $\gamma > 0$ and θ is sufficiently small, lower bound $z_{LR}(\lambda)$ will increase.

- Small θ : slow convergence
- Large θ : unstable

Subgradient, Held and Karp method

Initially

$$\lambda^{(0)} = \{0, \dots, 0\}$$

compute the new multipliers by recursion

$$\lambda_i^{(k+1)} := \begin{cases} \lambda_i^{(k)} & \text{if } |\gamma_i| \leq \epsilon \\ \max(\lambda_i^{(k)} + \theta \gamma_i, 0) & \text{if } |\gamma_i| > \epsilon \end{cases}$$

where γ is subgradient.

The step size is $\theta = \mu \frac{\bar{z} - \underline{z}}{\sum_i \gamma_i^2}$ where μ is a constant (e.g. 1)
(μ is halved if lower bound not increased in 20 iterations)

Iteration 0

We use a **simplified scheme** $\theta = 1/k$ for iteration $k \geq 1$

Subgradient

$$\gamma = Dx' - d = \left\{ \sum_k x_{ij}^k - u_{ij} \right\} = \{10 - 5, 20 - 10\} = \{5, 10\}$$

Recursion

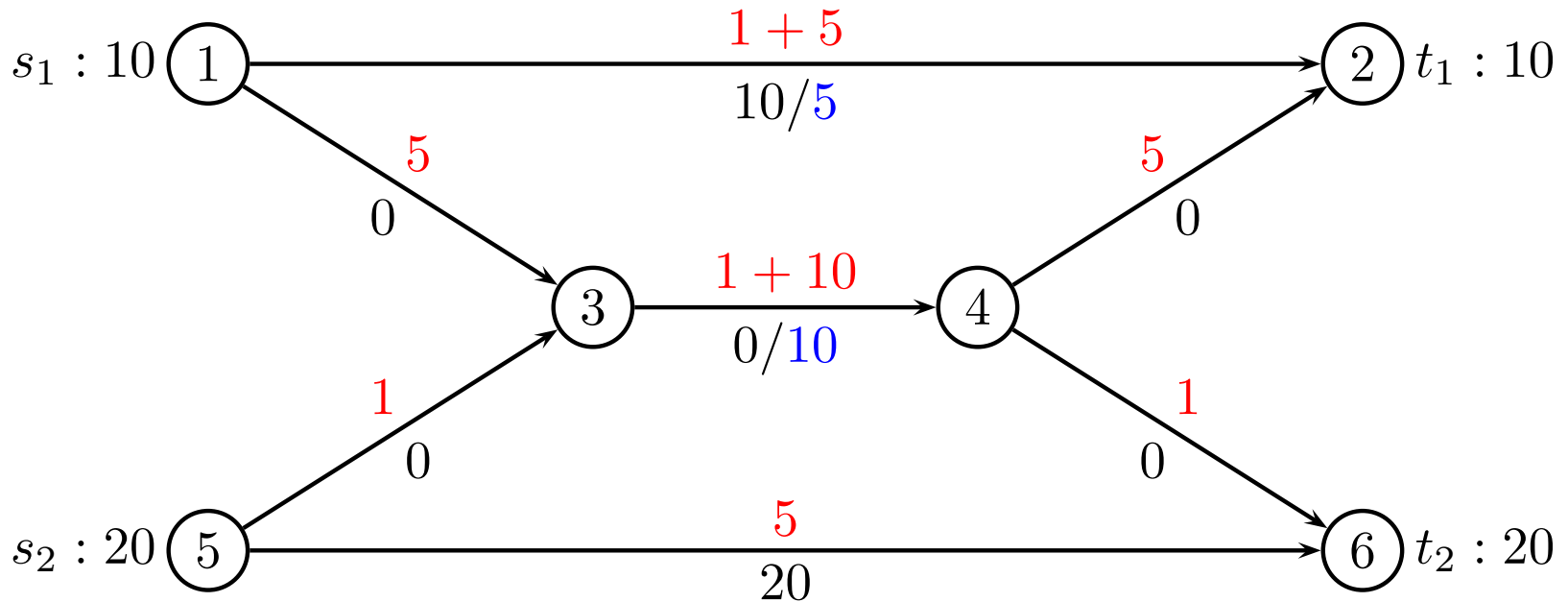
$$(\lambda_{12}, \lambda_{34}) = (\lambda_{12} + \theta \cdot 5, \lambda_{34} + \theta \cdot 10) = (0 + \theta \cdot 5, 0 + \theta \cdot 10)$$

Using $\theta = 1$ for $k = 0$ we get

$$(\lambda_{12}, \lambda_{34}) = (5, 10)$$

Iteration 1

$$\lambda_{12} = 5, \lambda_{34} = 10$$



flow cost: $10 \cdot (1 + 5) + 20 \cdot (5) = 160$

constant: $5\lambda_{12} + 10\lambda_{34} = 125$

lower bound: 35

Iteration 1

Subgradient

$$\gamma = \left\{ \sum_k x_{ij}^k - u_{ij} \right\} = \{10 - 5, 0 - 10\} = \{5, -10\}$$

Recursion

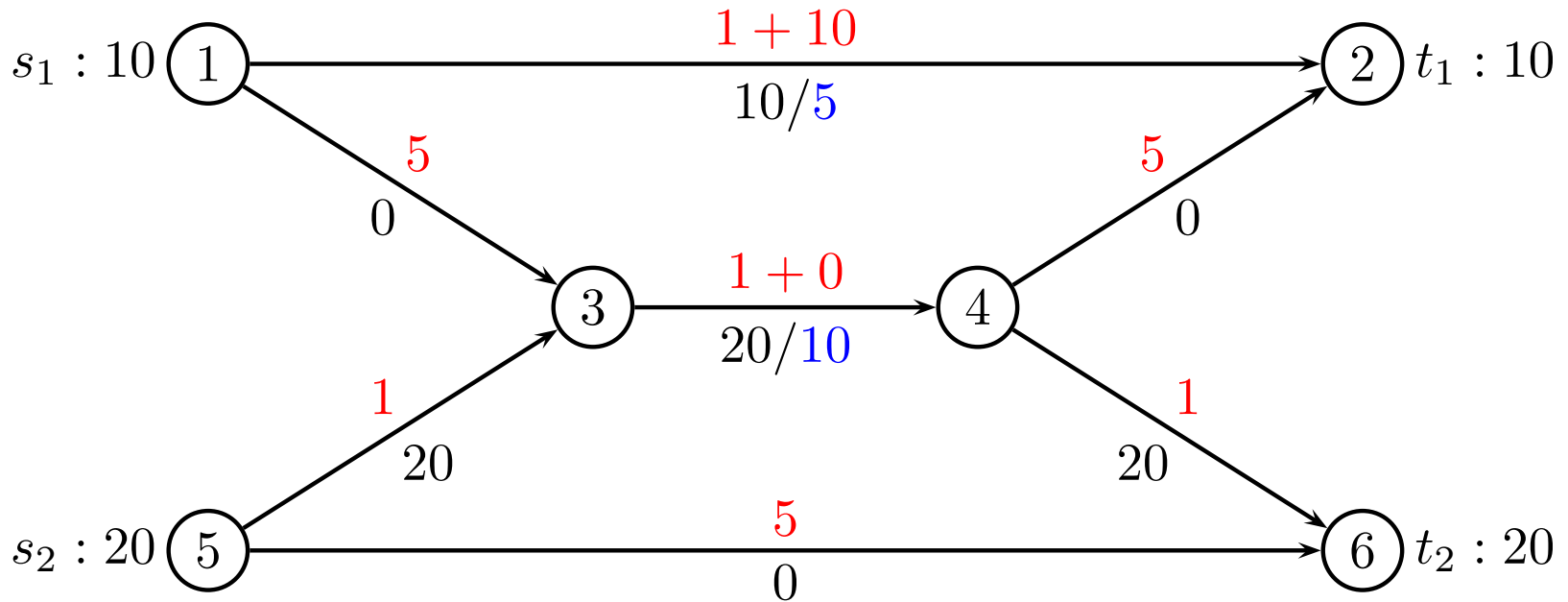
$$(\lambda_{12}, \lambda_{34}) = (5 + \theta \cdot 5, 10 - \theta \cdot 10)$$

Having $\theta = 1$ we get

$$(\lambda_{12}, \lambda_{34}) = (10, 0)$$

Iteration 2

$$\lambda_{12} = 10, \lambda_{34} = 0$$



flow cost: $10 \cdot (1 + 10) + 20 \cdot (1 + 1 + 1) = 170$

constant: $5\lambda_{12} + 10\lambda_{34} = 50$

lower bound: 120

Iteration 2

Subgradient

$$\gamma = \left\{ \sum_k x_{ij}^k - u_{ij} \right\} = \{10 - 5, 20 - 10\} = \{5, 10\}$$

Recursion

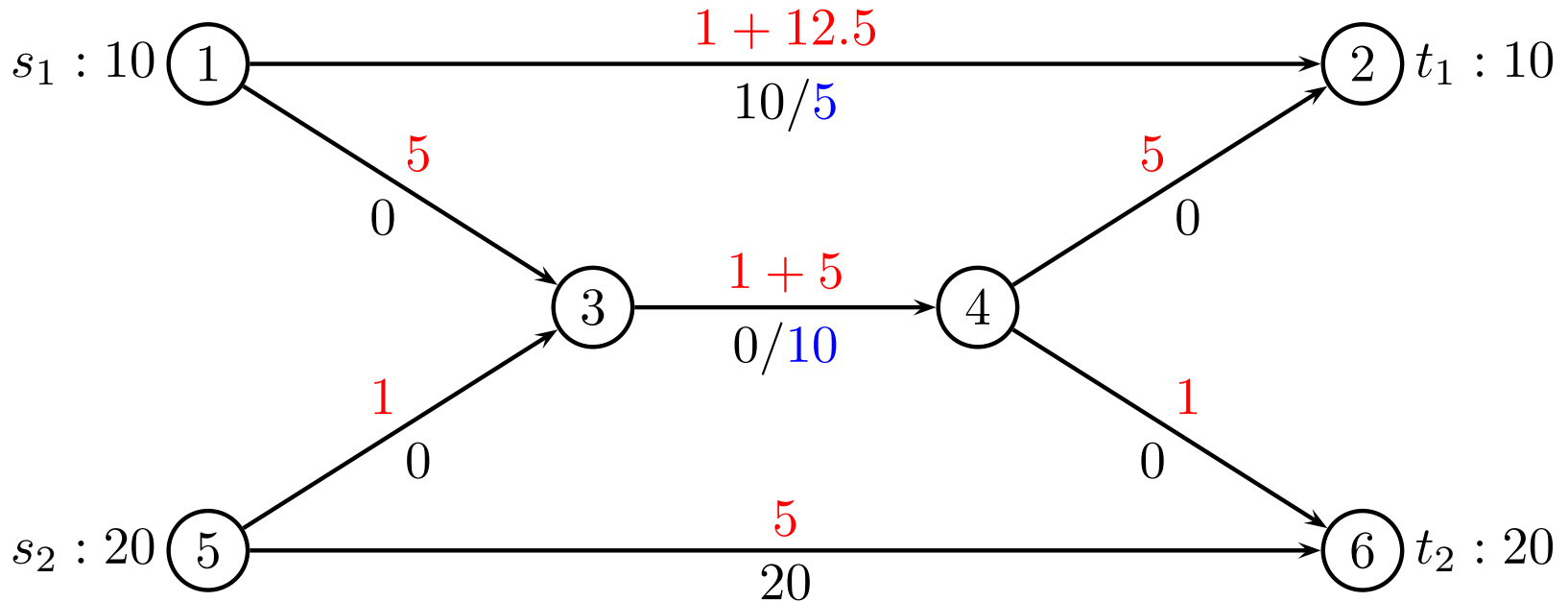
$$(\lambda_{12}, \lambda_{34}) = (10 + \theta \cdot 5, 0 + \theta \cdot 10)$$

Choosing $\theta = 1/2$ we get

$$(\lambda_{12}, \lambda_{34}) = (12.5, 5)$$

Iteration 3

$$\lambda_{12} = 12.5, \lambda_{34} = 5$$



flow cost: $10 \cdot (1 + 12.5) + 20 \cdot (5) = 235$

constant: $5\lambda_{12} + 10\lambda_{34} = 112.5$

lower bound: **122.5**

Iteration 3

Subgradient

$$\gamma = \left\{ \sum_k x_{ij}^k - u_{ij} \right\} = \{10 - 5, 0 - 10\} = \{5, -10\}$$

Recursion

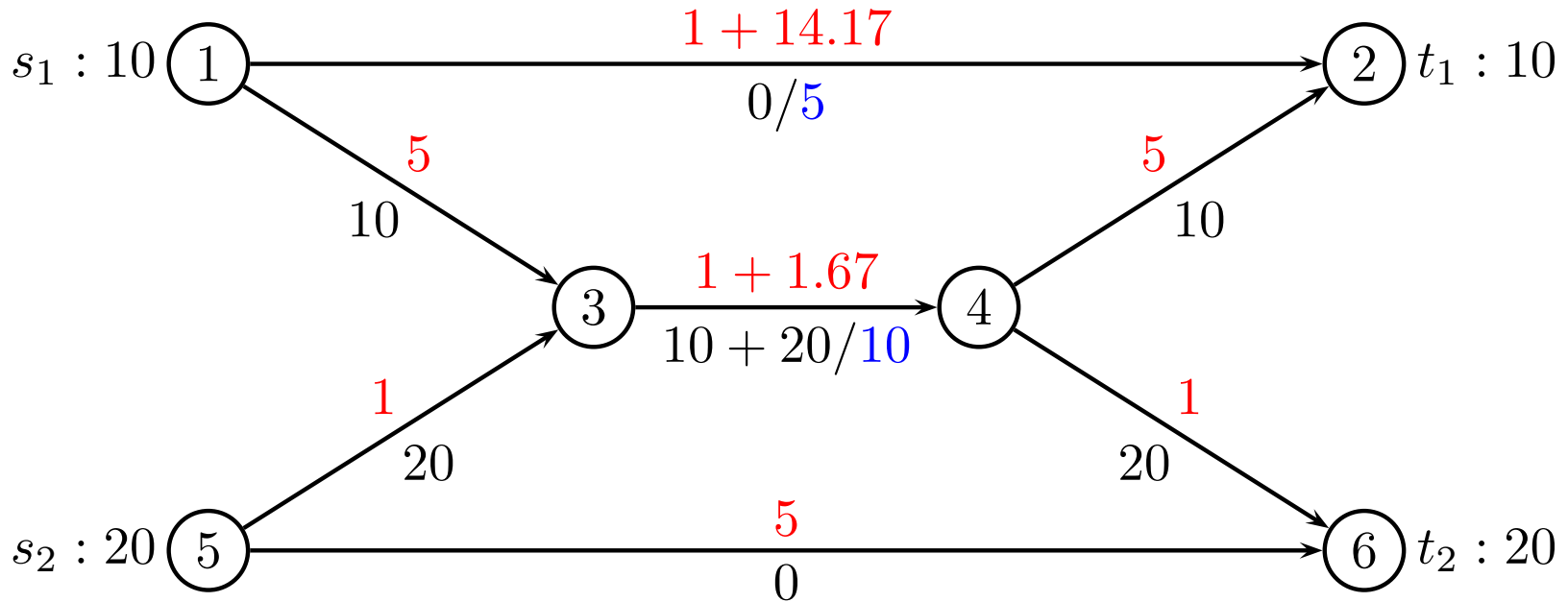
$$(\lambda_{12}, \lambda_{34}) = (12.5 + \theta \cdot 5, 5 - \theta \cdot 10)$$

Choosing $\theta = 1/3$ we get

$$(\lambda_{12}, \lambda_{34}) = (14.17, 1.67)$$

Iteration 4

$$\lambda_{12} = 14.17, \lambda_{34} = 1.67$$



flow cost: $10 \cdot (5 + 1 + 1.67 + 5) + 20 \cdot (1 + 1 + 1.67 + 1) = 220$

constant: $5\lambda_{12} + 10\lambda_{34} = 87.5$

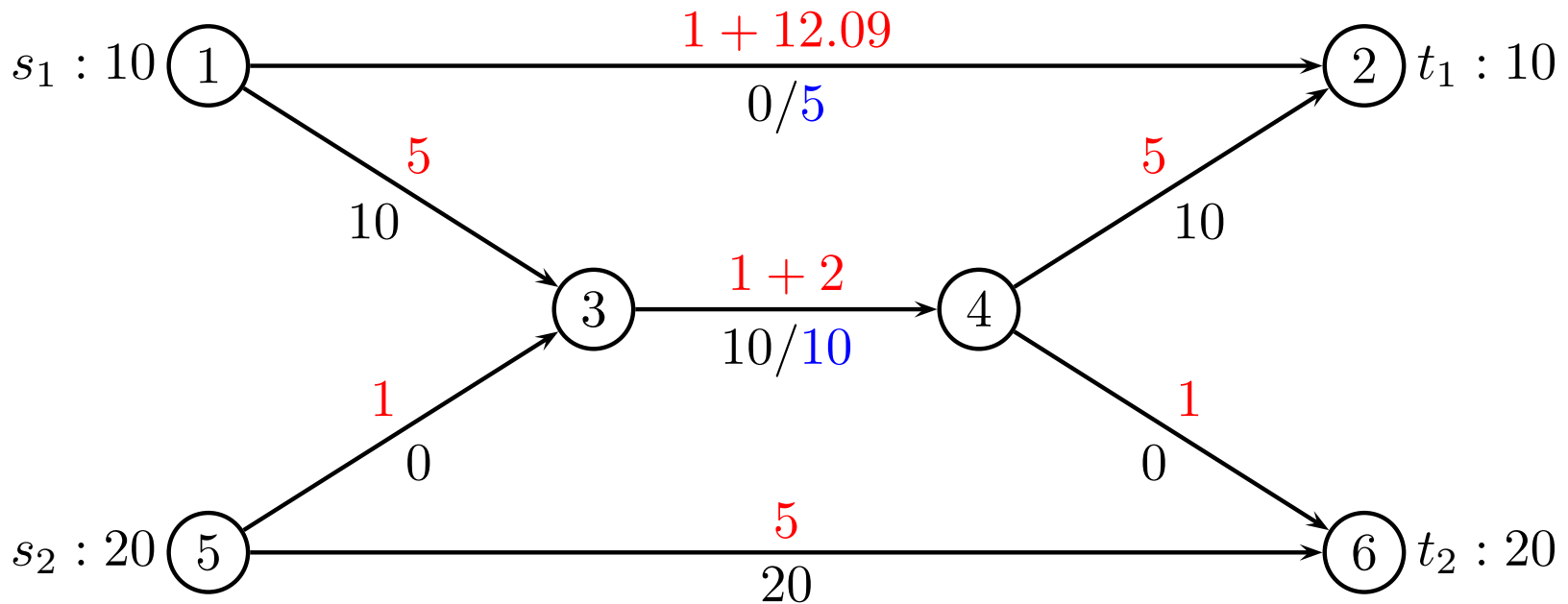
lower bound: 132.5

Iteration 5,...,13

calculating ...

Iteration 14

$$\lambda_{12} = 12.09, \lambda_{34} = 2$$



flow cost: $10 \cdot (5 + 3 + 5) + 20 \cdot (5) = 230$

constant: $5\lambda_{12} + 10\lambda_{34} = 80.45$

lower bound: 149.5

Greedy heuristic

For each iteration of λ run a greedy heuristic:

for each commodity $k \in K$

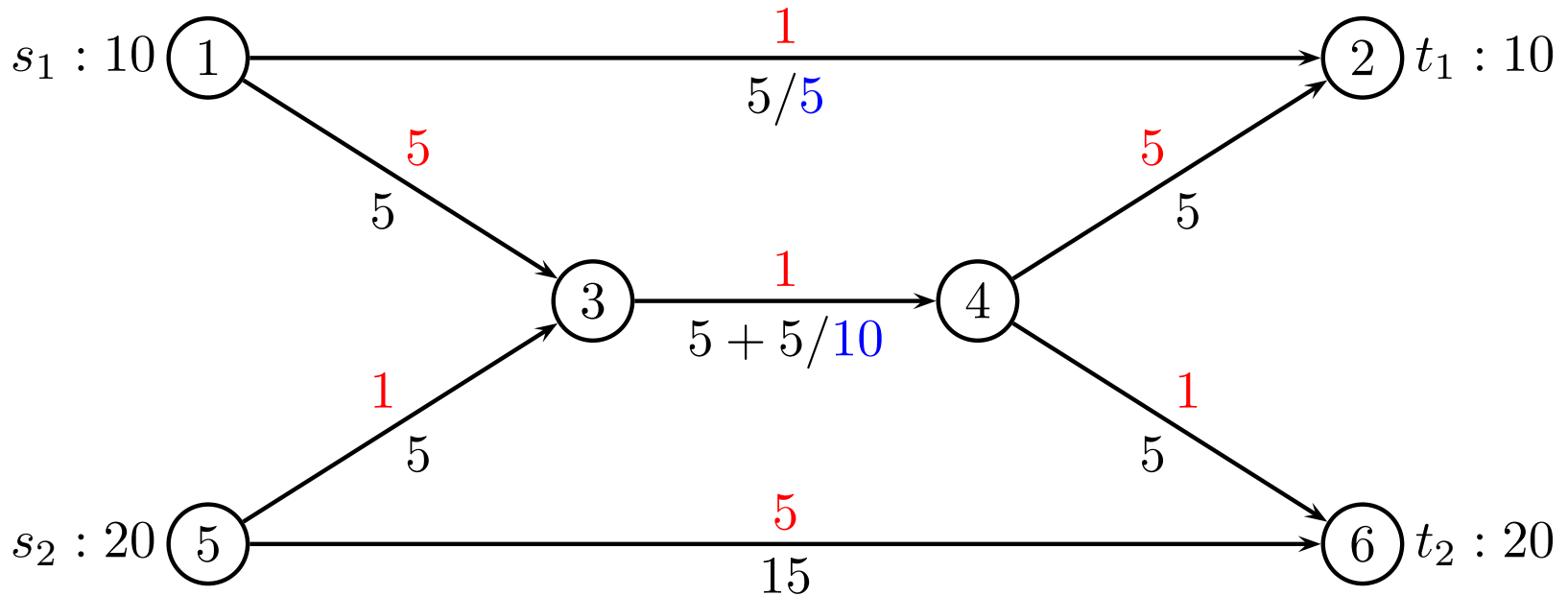
- 1 Find shortest path wrt. Lagrangian costs
- 2 Send as many units as residual capacity allows
- 3 If still demand left, remove filled edges, go to step 1

Greedy heuristic gives upper bound

if (upper bound = lower bound) **stop**

Heuristic solution

Iteration 0: $\lambda_{12} = 0$, $\lambda_{34} = 0$

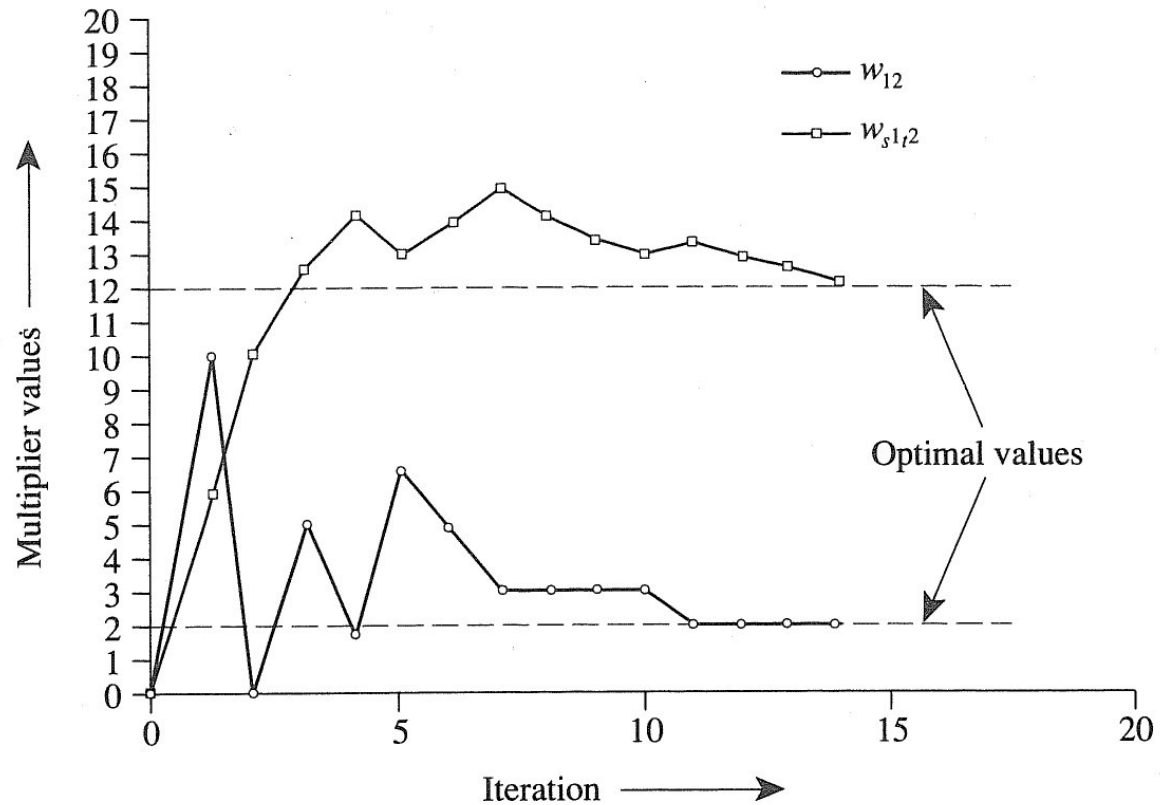


Cost: $5 \cdot 1 + 5 \cdot 5 + 10 \cdot 1 + 5 \cdot 5 + 5 \cdot 1 + 5 \cdot 1 + 15 \cdot 5 = 150$

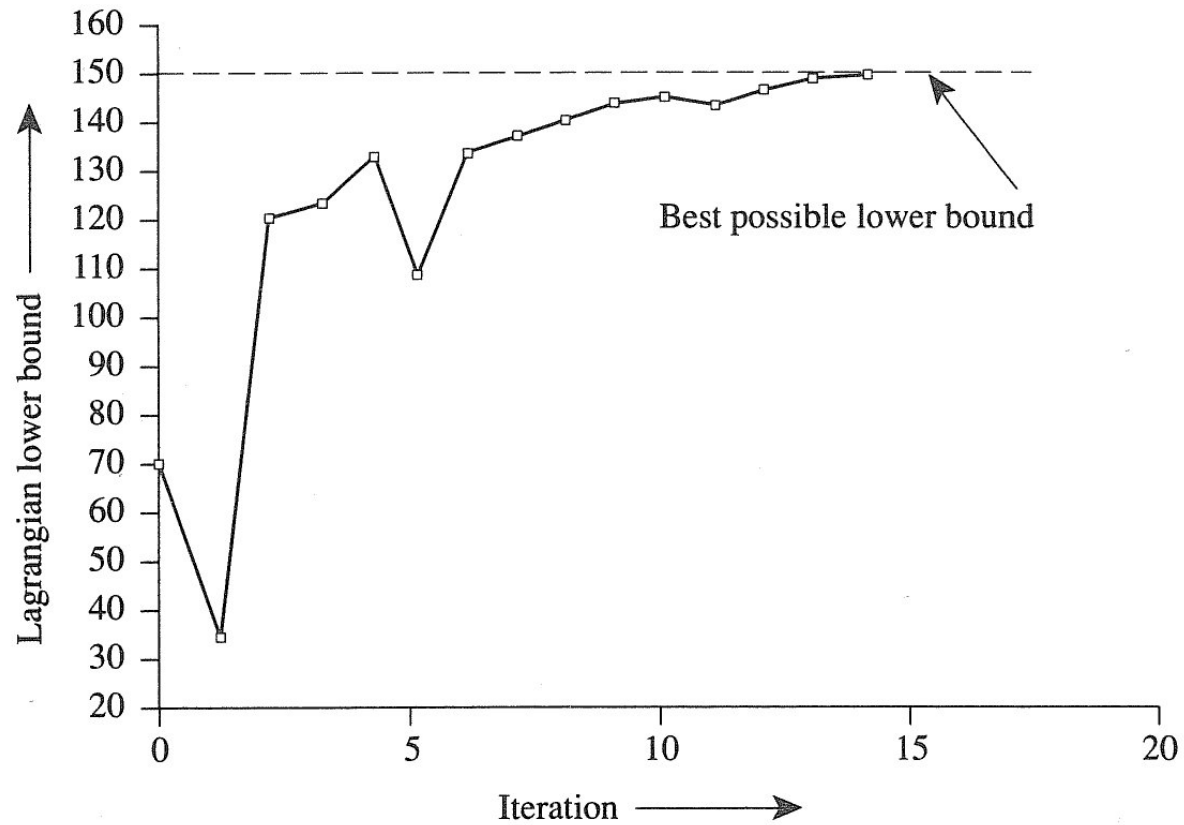
Lagrange iterations: Complete table

Iteration number q	w_{12}^q	$w_{s^1 t^1}^q$	Shortest paths	Shortest path costs	$10w_{12}^q + 5w_{s^1 t^1}^q$	Lower bound $L(w^q)$	θ_q
0	0	0	s^1-t^1 $s^2-1-2-t^2$	10(1) 20(1 + 1 + 1)	0	70	1
1	10	5	s^1-t^1 s^2-t^2	10(1 + 5) 20(5)	125	35	1
2	0	10	s^1-t^1 $s^2-1-2-t^2$	10(1 + 10) 20(1 + 1 + 1)	50	120	0.5
3	5	12.5	s^1-t^1 s^2-t^2	10(1 + 12.5) 20(5)	112.5	122.5	0.333
4	1.67	14.17	$s^1-1-2-t^1$ $s^2-1-2-t^2$	10(5 + 2.67 + 5) 20(1 + 2.67 + 1)	87.55	132.5	0.25
5	6.67	12.92	s^1-t^1 s^2-t^2	10(1 + 12.92) 20(5)	131.3	107.9	0.2
6	4.67	13.92	s^1-t^1 s^2-t^2	10(1 + 13.92) 20(5)	116.3	132.9	0.167
7	3	14.75	$s^1-1-2-t^1$ s^2-t^2	10(5 + 4 + 5) 20(5)	103.8	136.3	0.143
8	3	14.04	$s^1-1-2-t^1$ s^2-t^2	10(5 + 4 + 5) 20(5)	100.2	139.8	0.125
9	3	13.41	$s^1-1-2-t^1$ s^2-t^2	10(5 + 4 + 5) 20(5)	97.05	143.0	0.111
10	3	12.86	s^1-t^1 s^2-t^2	10(1 + 12.86) 20(5)	94.3	144.3	0.1
11	2	13.36	$s^1-1-2-t^1$ s^2-t^2	10(5 + 3 + 5) 20(5)	86.8	143.2	0.091
12	2	12.9	$s^1-1-2-t^1$ s^2-t^2	10(5 + 3 + 5) 20(5)	84.5	145.5	0.083
13	2	12.48	$s^1-1-2-t^1$ s^2-t^2	10(5 + 3 + 5) 20(5)	82.2	147.6	0.077
14	2	12.09	$s^1-1-2-t^1$ s^2-t^2	10(5 + 3 + 5) 20(5)	80.25	149.5	0.071

Lagrange multipliers



Lower bound



Lagrangian relaxation - wrapping up

Scheme for step size θ

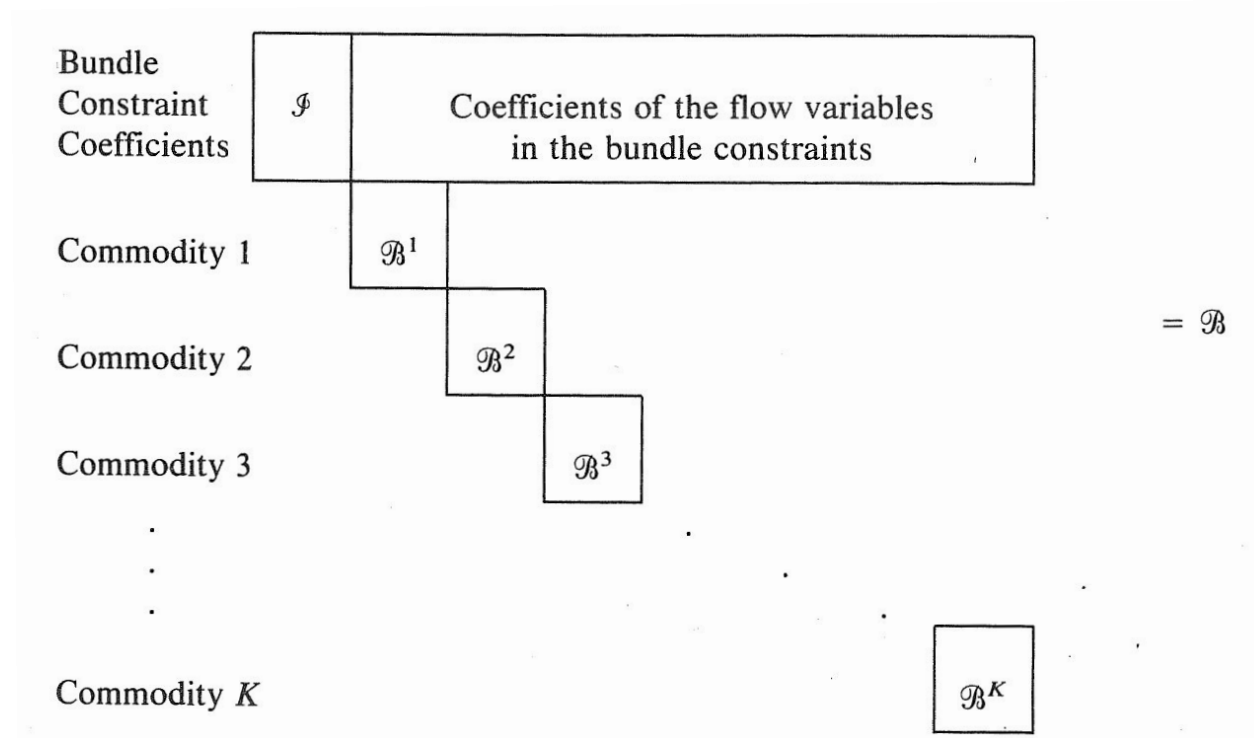
- Small problems, use $1, 1, \frac{1}{2}, \frac{1}{3}, \frac{1}{4}, \frac{1}{5}, \dots$
- Big problems, use Held and Karp

Stopping criteria

- In every iteration check if solution is feasible, and possibly update upper bound z
- If lower bound is equal to upper bound, stop
- If 20 iterations with no improvement in lower bound, decrease θ
- If 100 iterations with no improvement in lower bound stop

When stopped, you have a “guess” for dual variables.
Convert dual to primal.

Dantzig-Wolfe versus Lagrangian



Both split problem into easier subproblems
 Column generation: improves upper bound
 Subgradient optimization: improves lower bound

Lagrangian relaxation

- It can be shown that best Lagrangian multipliers λ are dual variables
- If we know λ we can find x by duality
- We could also solve LP-dual to find best λ
- As LP problems can have multiple solutions, so can Lagrangian dual

Generally

- We can Lagrangian relax all constraints and use subgradient to find dual variables, i.e. solve LP problem
- Alternative method to Simplex, not searching corner points
- Main idea in interior point methods for LP